SOCIAL MESSAGING PLATFORM

Complete Product & Technical Documentation

PRD | User Flows | UI/UX | ERD | Database | API | Backend Architecture | Roadmap

Version 1.0 | 31 October 2026
Web-first | NestJS | PostgreSQL | Socket.IO
Light + Dark Theme

Document Control

Document	Complete Product & Technical Documentation
Version	1.0
Date	31 October 2026
Status	Baseline specification for review and implementation planning
Platform strategy	Web first; mobile clients later
Backend	NestJS + PostgreSQL + Socket.IO
Architecture	Modular Monolith, PNPM Monorepo
Design	Light + Dark

Contents

- 1. Executive Summary
- 2. Product Vision and Locked Decisions
- 3. Goals, Non-goals, and Success Metrics
- 4. Users, Roles, and Core Use Cases
- 5. Functional Requirements and Acceptance Criteria
- 6. User Flows
- 7. Information Architecture and UI/UX
- 8. System Architecture
- 9. Monorepo Folder Structure
- 10. Database Design
- 11. API Contract
- 12. Socket.IO Realtime Contract
- 13. Media Upload and Processing
- 14. Security, Privacy, and Abuse Prevention
- 15. Non-functional Requirements and Reliability
- 16. Testing and Quality Assurance
- 17. Development Roadmap and Milestones
- 18. Deployment and Operations
- 19. Acceptance Criteria for v1
- 20. Open Decisions Before Implementation
- Appendix A. Companion Files
- Appendix B. Glossary

1. Executive Summary

This document defines the product, user experience, system architecture, database baseline, API contracts, security model, development roadmap, and operational plan for a chat-first social networking platform.

The product combines private real-time messaging with permanent friend-only posts and short videos. It is not a public social network and does not include a public content feed or stranger recommendation engine.

- Core promise: Connect with accepted friends, talk in real time, and share meaningful moments.
- Initial platform: Web application.
- Future platform: Android and iOS clients using the same backend APIs and real-time gateway.
- Architecture: Modular Monolith in a PNPM monorepo.
- Backend: NestJS + REST + Socket.IO.
- Data: PostgreSQL as the system of record; Redis for transient state and scaling support.
- Media: Private object storage with signed access URLs.

2. Product Vision and Locked Decisions

2.1 Product vision

Create a private, friend-based communication space where messaging is the primary activity and social sharing strengthens existing relationships.

2.2 Locked product decisions

Area	Decision
Connection	Mutual friendship. A friend request must be accepted before direct chat and friends-only content access are enabled.
Moments	Permanent posts and short videos. Ephemeral 24-hour Stories are not part of v1.
Discovery	Friends-only content feed. No public feed, stranger recommendations, or interest-based discovery.
Platform	Web first; mobile later.
Backend	NestJS + PostgreSQL + Socket.IO.
Design	Light and Dark themes.
Product priority	Chat-first; Moments are secondary to communication.

2.3 Discovery clarification

The platform may provide exact-username lookup so a user can find a person they already know and send a friend request. This is a connection utility, not a people-discovery feed. Search results must expose only a limited profile preview until the friendship is accepted.

3. Goals, Non-goals, and Success Metrics

3.1 Goals

- Make one-to-one messaging fast, dependable, and easy to reach.
- Allow users to establish mutual friendships through explicit requests.
- Allow accepted friends to share permanent text/photo posts and short videos.
- Protect conversations and social content with server-side authorization.
- Keep the backend reusable by future mobile clients.

3.2 Non-goals for v1

- Public posts or public profile feeds.
- Stranger discovery, people recommendations, or follow-based networking.
- Ephemeral Stories or disappearing posts.

- Live streaming, marketplace, payments, or creator monetization.
- Audio/video calling in the first release.
- End-to-end encrypted messaging in the first release; this requires a separate cryptographic design.

3.3 Success metrics

Metric	Definition / purpose
Daily active users	Unique users with meaningful activity per day.
Meaningful conversations per active user	Distinct conversations with at least one user-authored message.
Message delivery success	Share of accepted messages delivered to intended recipients.
Friend request acceptance rate	Accepted requests divided by resolved requests.
Weekly retention	Users returning in the following week.
Crash-free sessions	Share of sessions without a client crash.

4. Users, Roles, and Core Use Cases

4.1 User profiles

- Regular communicator: primarily uses direct messages and group conversations when enabled.
- Social sharer: publishes photos, text updates, and short videos for accepted friends.
- Private user: keeps a small friend list and expects strict privacy controls.

4.2 Roles

Role	Capabilities
Guest	Register, log in, verify account, reset password.
Authenticated user	Manage profile, search exact usernames, send/receive requests, chat with accepted friends, publish and view eligible posts.
Conversation member	Read and send messages according to conversation membership and friendship policy.
Group admin (future phase)	Manage group metadata and membership under the approved group policy.
Moderator	Review reports and take documented moderation actions.
System administrator	Operate infrastructure and account-support workflows; privileged access must be audited.

4.3 Core use cases

1. Register and create a profile.
2. Find a known person by exact username and send a friend request.
3. Accept or reject an incoming friend request.
4. Open or create a direct conversation with an accepted friend.
5. Send text and supported media messages.
6. See delivery/read state and recover missed messages after reconnecting.
7. Publish a permanent text, photo, or short-video post.
8. View a chronological feed containing own posts and accepted friends' posts.
9. React to and comment on accessible posts.
10. Block, unfriend, report, or manage privacy settings.

5. Functional Requirements and Acceptance Criteria

5.1 Authentication and account management

- FR-AUTH-01: Users can register with a unique username and at least one verified contact method, subject to the final registration policy.
- FR-AUTH-02: Passwords are hashed using Argon2id; plaintext passwords are never stored.

- FR-AUTH-03: Users can log in, log out, refresh sessions, and revoke sessions.
- FR-AUTH-04: Users can request password reset through a verified contact method.
- FR-AUTH-05: Users can deactivate or request deletion of their account.

Acceptance: duplicate usernames/contact identifiers are rejected; invalid credentials do not reveal whether an account exists; revoked refresh tokens cannot create new sessions.

5.2 Profiles

- FR-PROF-01: Every account has one profile with display name and username.
- FR-PROF-02: Users can update bio, avatar, and cover image.
- FR-PROF-03: Profile fields have length and file constraints.
- FR-PROF-04: Non-friends see only the minimum profile preview needed for exact-username friend requests.

5.3 Friendships

- FR-FRIEND-01: A user can search by exact username.
- FR-FRIEND-02: A user can send one pending request per user pair.
- FR-FRIEND-03: The recipient can accept or reject the request.
- FR-FRIEND-04: Acceptance creates one canonical friendship in the same database transaction.
- FR-FRIEND-05: Users can cancel outgoing requests, unfriend, block, and unblock.
- FR-FRIEND-06: A block prevents new friend requests and direct messages between the affected users.

Acceptance: reverse-direction duplicate pending requests are prevented; users cannot send requests to themselves; a blocked relationship is checked in both directions before creating a new friendship or conversation.

5.4 Direct messaging

- FR-MSG-01: Only accepted friends can create or send messages in a direct conversation.
- FR-MSG-02: Supported v1 message types are text and approved media attachments; voice messages may be included after media validation is implemented.
- FR-MSG-03: Messages are persisted before the server broadcasts the new-message event.
- FR-MSG-04: Clients can retrieve message history with cursor pagination.
- FR-MSG-05: The system supports delivery and read receipts.
- FR-MSG-06: Users can reply to messages; edit/delete behavior follows a documented time and retention policy.
- FR-MSG-07: Client-generated message IDs prevent duplicates during retries.

Acceptance: a non-friend cannot send a direct message by guessing a conversation ID; a message remains available after recipient reconnects; a retried client message does not create a duplicate row.

5.5 Group messaging

Group messaging is planned for a later phase. Before implementation, finalize who may add members, whether every participant must be a friend of the creator, what happens when members unfriend or block one another, and how historical messages are retained for removed members.

5.6 Moments and feed

- FR-POST-01: Users can create permanent text, photo, video, or mixed posts.
- FR-POST-02: The default and initial supported audience is friends_only.
- FR-POST-03: The feed contains the user's own posts and posts from accepted friends only.
- FR-POST-04: Feed order is chronological, newest first; no recommendation ranking is used in v1.
- FR-POST-05: Users can edit their own caption and delete their own posts.
- FR-POST-06: Eligible friends can react and comment.
- FR-POST-07: Unfriending revokes future access to friends-only content.

Acceptance: a non-friend cannot retrieve a post by directly requesting its UUID; removing a friendship removes that author's posts from the viewer's future feed queries.

5.7 Notifications

- FR-NOTIF-01: In-app notifications cover friend requests, accepted requests, new messages, reactions, and comments.
- FR-NOTIF-02: Users can mark notifications as read and manage supported preferences.
- FR-NOTIF-03: Push notification delivery is asynchronous and retryable.

5.8 Moderation and privacy

- FR-PRIV-01: Users can block and report accounts or content.
- FR-PRIV-02: Authorization is enforced in backend services, not only hidden in the UI.
- FR-PRIV-03: Private media is served through short-lived signed URLs after access checks.
- FR-PRIV-04: Sensitive administrative actions are auditable.

6. User Flows

6.1 Registration

1. Open the web application and choose Sign up.
2. Enter username, password, display name, and the selected contact method.
3. Complete email or phone verification when required.
4. Create the profile and initial settings.
5. Enter the Chats screen.

6.2 Friend request

1. Open Friends and choose Add friend.
2. Enter an exact username.
3. Review the limited profile preview.
4. Send the request.
5. Recipient receives an in-app/push notification.
6. Recipient accepts or rejects.
7. On acceptance, the backend creates the canonical friendship transactionally and enables direct chat and friend-only post visibility.

6.3 Direct chat

1. Open Chats and select an accepted friend.
2. Client requests the direct conversation; backend returns the unique conversation or creates it idempotently.
3. Client loads recent messages with cursor pagination.
4. User sends a message with a client_message_id.
5. Server authenticates, checks friendship/block/membership, persists message and outbox event in a transaction.
6. Event publisher emits message:new to the recipient's authorized user room.
7. Recipient acknowledges delivery; read state is updated when the conversation is viewed.

6.4 Create a permanent Moment

1. Open Moments and select Create post.
2. Choose text, photo, video, or mixed content.
3. For media, create an upload intent and upload directly to private object storage.
4. Complete upload; media processing validates and prepares the asset.
5. Write caption and preview the post.
6. Publish with friends_only visibility.

7. Post appears on the author's profile and in accepted friends' chronological feeds.

6.5 Unfriend and block

- Unfriend removes the canonical friendship and revokes access to friends-only posts. Existing direct conversation history may remain visible, but new direct messages are disabled until friendship is restored.
- Block prevents new direct communication and friend requests in both directions. The application should hide or restrict the blocked user's profile and content according to the privacy policy.

7. Information Architecture and UI/UX

7.1 Primary navigation

Navigation item	Purpose
Chats	Conversation list, unread state, search, new chat.
Moments	Chronological feed of own and friends' permanent posts and short videos.
Friends	Friend list, incoming/outgoing requests, exact-username lookup.
Profile	Personal profile, posts, edit profile, settings.

7.2 Core screens

- Authentication: welcome, register, login, verification, password reset.
- Chats: conversation list, search, unread counts, latest message preview, online indicator.
- Conversation: header, message history, composer, attachment picker, delivery/read state, typing indicator.
- Moments: create-post entry point, post cards, video player, reactions, comments.
- Create post: text editor, media selection, upload progress, preview, publish.
- Friends: requests, friend list, exact username search, profile preview.
- Profile: avatar, cover, display name, username, bio, friend count, posts.
- Settings: theme, privacy, notification preferences, sessions, account actions.

7.3 Responsive web layout

Desktop chat layout should use a conversation sidebar and an active conversation panel. On narrow screens, the conversation list and active conversation become separate navigable views. Moments should use a readable single-column feed with responsive media sizing.

7.4 Theme system

- Support Light, Dark, and System preference.
- Use semantic design tokens for background, surface, text, muted text, border, primary, success, warning, and destructive states.
- Do not hard-code theme colors inside feature components.
- Persist user preference in user_settings and apply it on initial render to reduce theme flash.
- Maintain accessible contrast and visible focus states.

7.5 UI states

- Loading: skeletons for conversation list, messages, and feed.
- Empty: explain how to add friends or start the first conversation.
- Error: actionable retry and non-technical message.
- Offline: show connection state and preserve unsent draft messages locally.
- Upload: show progress, processing, success, and retry/failure states.

8. System Architecture

8.1 Architecture style

Use a Modular Monolith for v1. Modules are separated by domain boundaries and communicate through application services and domain events. Avoid microservices until measured scale or independent deployment needs justify the operational cost.

8.2 Technology decisions

Layer	Technology	Responsibility
Web	Next.js + TypeScript	Responsive browser client.
UI	Tailwind CSS + shadcn/ui	Reusable accessible interface components.
Server state	TanStack Query	API cache, pagination, invalidation.
Client state	Zustand	UI state, drafts, transient client state.
Backend	NestJS + TypeScript	Business logic and authorization.
API	REST /api/v1	Stable cross-client contract.
Realtime	Socket.IO	Live message and presence events.
Database	PostgreSQL	Durable relational source of truth.
ORM	Prisma	Typed data access and migrations; raw SQL for specialized indexes.
Cache / adapter	Redis	Presence, rate limits, Socket.IO scaling, queues.
Media	Private S3-compatible storage	Images, video, voice, attachments.
Jobs	BullMQ or equivalent	Media processing and notification delivery.
Deployment	Docker	Repeatable local and production runtime.

8.3 Request flow

Browser -> Next.js client -> REST API or authenticated Socket.IO connection -> NestJS guards and policies -> domain service -> Prisma/PostgreSQL. Media bytes travel directly from the browser to object storage using a short-lived signed upload URL.

8.4 Backend module boundaries

- Auth: credentials, verification, refresh sessions, logout.
- Users/Profiles: account and profile data.
- Friendships: requests, accepted friendship, blocks.
- Conversations: direct conversation lifecycle and membership.
- Messages: persistence, receipts, reactions, history.
- Media: upload intents, asset state, signed URLs, processing.
- Posts: post lifecycle and friends-only feed.
- Reactions/Comments: post interactions.
- Notifications: in-app records and delivery orchestration.
- Realtime: socket authentication, rooms, event transport, presence.
- Moderation: reports, review status, enforcement actions.
- Infrastructure: database, Redis, queue, storage, logging.

8.5 Dependency rules

- Controllers and gateways are transport adapters; they should not contain business rules.
- Services own business use cases and authorization decisions.
- A module must not write another module's tables directly.
- Cross-module work should call an exported application service or publish a domain event.
- Realtime gateways call message/conversation services; they do not duplicate persistence logic.

9. Monorepo Folder Structure

Use PNPM workspaces and optionally Turborepo for task orchestration. The detailed tree is provided in architecture/project-folder-structure.txt.

```
social-platform/
■■■ apps/
■   ■■■ web/                        # Next.js + TypeScript
■   ■   ■■■ src/
■   ■       ■■■ app/                # routes and layouts
■   ■       ■■■ components/         # reusable UI
■   ■       ■■■ features/           # chat, friends, moments, profile
■   ■       ■■■ hooks/
■   ■       ■■■ stores/             # Zustand client state
■   ■       ■■■ lib/                # API, socket, auth utilities
■   ■       ■■■ styles/             # theme tokens
■   ■■■ api/                        # NestJS
■       ■■■ src/
■           ■■■ common/             # guards, filters, pipes, decorators
■           ■■■ config/
■           ■■■ database/           # Prisma service and migrations
■           ■■■ infrastructure/     # Redis, storage, queue, logging
■           ■■■ modules/
■           ■   ■■■ auth/
■           ■   ■■■ users/
■           ■   ■■■ profiles/
■           ■   ■■■ friendships/
■           ■   ■■■ conversations/
■           ■   ■■■ messages/
■           ■   ■■■ media/
■           ■   ■■■ posts/
■           ■   ■■■ reactions/
■           ■   ■■■ comments/
■           ■   ■■■ notifications/
■           ■   ■■■ devices/
■           ■   ■■■ moderation/
■           ■   ■■■ realtime/
■           ■   ■■■ health/
■           ■■■ main.ts
■■■ packages/
■   ■■■ api-client/                 # generated typed client
■   ■■■ config/                     # shared lint/tsconfig
■■■ database/
■   ■■■ schema.sql
■   ■■■ erd.mmd
■■■ infra/
■   ■■■ docker/
■   ■■■ nginx/
■   ■■■ scripts/
■■■ docs/
■   ■■■ product/
■   ■■■ architecture/
■   ■■■ database/
■   ■■■ api/
■■■ .github/workflows/
■■■ pnpm-workspace.yaml
■■■ turbo.json
■■■ package.json
■■■ README.md

Module internal pattern (use where complexity warrants):
module-name/
■■■ module-name.module.ts
■■■ controllers/
■■■ gateways/        # only for realtime-owned gateways
■■■ services/
■■■ dto/
■■■ policies/
■■■ repositories/    # for complex queries/data access
■■■ events/
■■■ tests/
```

9.1 Module internal pattern

```
module-name/
  module-name.module.ts
  controllers/
  gateways/        # only when the module owns socket transport
  services/
  dto/
  policies/
  repositories/    # for complex query/data access boundaries
  events/
  tests/
```

Do not create empty folders for every possible abstraction. Introduce repositories, domain events, and gateways where they provide clear separation.

10. Database Design

10.1 Database principles

- PostgreSQL is the source of truth for users, friendships, conversations, messages, posts, and notifications.
- Use UUID primary keys and timestamptz timestamps.
- Use foreign keys, unique constraints, check constraints, and indexes for durable invariants.

- Use soft deletion for user-generated content where retention/moderation requires it; define actual retention periods before launch.
- Store media metadata in PostgreSQL and binary media in private object storage.
- Use cursor pagination for messages, feed, comments, and notifications.

10.2 Main tables

Domain	Tables	Purpose
Identity	users, user_profiles, user_settings	Account, profile, preferences.
Friendship	friend_requests, friendships, blocks	Mutual connection and restrictions.
Messaging	conversations, direct_conversation_pairs, conversation_members, messages, message_attachments, message_receipts, message_reactions	Direct/group conversation and message history.
Media	media_assets	Private media metadata and processing lifecycle.
Social	posts, post_media, post_reactions, post_comments	Permanent posts and interactions.
Notifications	notifications, devices	In-app and push notification records.
Security	user_sessions, reports	Refresh sessions and abuse reports.
Reliability	outbox_events	Reliable asynchronous event publication.

10.3 Canonical friendship

Store each friendship once using user_low_id and user_high_id, where user_low_id < user_high_id. The pair is the primary key. This avoids duplicate A-B and B-A rows.

10.4 Direct conversation uniqueness

Use direct_conversation_pairs with a canonical user pair and a unique constraint. Creating a direct conversation is an idempotent transaction: validate friendship and blocks, insert or retrieve the pair, then ensure the two active members exist.

10.5 Message reliability

Persist a message and its outbox event in one database transaction. A background publisher reads unprocessed outbox rows and emits the event. Client-generated message IDs make retries idempotent.

10.6 Feed query

For the authenticated viewer, retrieve own posts plus posts authored by accepted friends, exclude deleted posts and blocked relationships, enforce friends_only visibility, sort by created_at DESC then id DESC, and paginate with a cursor. Do not expose a public feed endpoint.

10.7 Schema files

The companion database/schema.sql is a baseline PostgreSQL schema. database/erd.mmd contains the Mermaid ER diagram source. Review both before applying in a production environment.

11. API Contract

11.1 API conventions

- Base path: /api/v1.
- Use JSON for normal request/response bodies.
- Use consistent error format: statusCode, code, message, requestId, details (safe validation details only).
- Use cursor pagination with limit and nextCursor; cap page size.
- Use ISO-8601 UTC timestamps.
- Document endpoints in Swagger/OpenAPI and generate a typed frontend client.

11.2 Endpoint groups

Group	Representative endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout
Users	GET /users/me, GET /users/search?username=..., GET /users/:id/preview
Profiles	PATCH /profiles/me, GET /profiles/:userId
Friendships	POST /friend-requests, GET /friend-requests/incoming, POST /friend-requests/:id/accept, POST /friend-requests/:id/reject, GET /friends, DELETE /friends/:userId
Conversations	GET /conversations, POST /conversations/direct, GET /conversations/:id
Messages	GET /conversations/:id/messages, PATCH /messages/:id, DELETE /messages/:id, POST /messages/:id/reactions
Media	POST /media/upload-intents, POST /media/:id/complete, GET /media/:id/access
Posts	POST /posts, GET /feed, GET /posts/:id, PATCH /posts/:id, DELETE /posts/:id
Interactions	POST /posts/:id/reactions, POST /posts/:id/comments, GET /posts/:id/comments
Notifications	GET /notifications, PATCH /notifications/:id/read, POST /notifications/read-all
Moderation	POST /reports, POST /users/:id/block, DELETE /users/:id/block

11.3 Example error response

```
{
  "statusCode": 403,
  "code": "FRIENDSHIP_REQUIRED",
  "message": "You can message this user after becoming friends.",
  "requestId": "req_...",
  "details": null
}
```

A starter OpenAPI outline is included at api/openapi-outline.yaml. It must be expanded with full schemas, authentication details, response bodies, and error codes before release.

12. Socket.IO Realtime Contract

12.1 Connection

Authenticate the Socket.IO handshake using a short-lived access credential. Do not accept userId values from the client as proof of identity. Derive the user identity from the validated session/token.

12.2 Rooms

- user:{userId}: private per-user events across the user's active devices.
- conversation:{conversationId}: conversation events, joined only after server-side membership authorization.

12.3 Events

Event	Direction	Purpose
message:send	Client -> Server	Submit a message with conversationId, clientMessageId, type, body/attachment references.
message:new	Server -> Client	Deliver a persisted message to authorized participants.
message:delivered	Server -> Client	Notify sender that a recipient device received the message.
message:read	Client -> Server / Server -> Client	Update and broadcast read position/receipt.
typing:start	Client -> Server -> Room	Transient typing state.
typing:stop	Client -> Server -> Room	Clear typing state.
presence:update	Server -> Client	Online/last-seen state changes subject to privacy settings.
conversation:update	Server -> Client	Conversation metadata or latest-message preview changed.

12.4 Message send payload

```
{
  "conversationId": "uuid",
  "clientMessageId": "uuid",
  "messageType": "text",
  "body": "Hello",
  "replyToMessageId": null
}
```

12.5 Server processing

1. Authenticate socket and validate payload.
2. Load conversation membership and confirm sender is an active member.
3. For direct conversations, confirm accepted friendship and no block in either direction.
4. Validate message type, body length, attachment ownership, and attachment readiness.
5. Insert message and outbox event in one transaction.
6. Acknowledge the sender with the canonical message ID.
7. Publish message:new to authorized recipient rooms.
8. Record delivery/read receipts as clients acknowledge.

12.6 Reconnect and recovery

Socket events are not the durable message store. On reconnect, the client requests messages after its last known cursor using REST. The server returns persisted records. Typing and presence are transient and may be lost during disconnect.

13. Media Upload and Processing

13.1 Upload lifecycle

1. Client requests an upload intent with media category, file name, MIME type, and expected size.
2. Backend authenticates the user, checks allowed type/size, creates a pending media_assets record, and returns a short-lived signed upload URL.
3. Client uploads directly to private object storage.
4. Client calls upload completion endpoint.
5. Worker verifies actual file signature, size, checksum, and malware policy.
6. Worker generates image thumbnails or video poster/transcoded renditions as applicable.
7. Asset status becomes ready or failed/quarantined.
8. Post/message creation can reference only ready assets owned by the user or explicitly authorized for that use.

13.2 Media security

- Never expose permanent public object URLs for private media.
- Use signed URLs with short expiry after checking post visibility or conversation membership.
- Validate MIME type using file contents, not only the browser-supplied header.
- Set file-size and duration limits in configuration.
- Keep original and derived assets private.
- Delete orphaned uploads through a scheduled cleanup job.

13.3 Proposed configurable limits

The initial short-video duration target is 60 seconds, subject to product approval and infrastructure testing. Image/video/voice file-size limits, allowed codecs, resolution, and retention periods must be configured and confirmed before launch.

14. Security, Privacy, and Abuse Prevention

14.1 Authentication

- Hash passwords with Argon2id and a reviewed parameter set.
- Use short-lived access tokens and rotating refresh tokens.
- Store refresh-token hashes in user_sessions.
- Use HttpOnly, Secure, SameSite cookies for browser refresh sessions and implement CSRF protection appropriate to the deployment.

- Support session revocation and device/session review.

14.2 Authorization

- Use deny-by-default authorization.
- Check friendship and block status on every direct-message create/send operation.
- Check conversation membership on history, attachments, receipts, and socket room joins.
- Check post visibility and friendship on feed, post detail, media access, reactions, and comments.
- Do not rely on hidden UI controls as security.

14.3 Privacy

- No public feed or public post visibility in v1.
- Exact-username lookup returns a limited profile preview.
- Online and last-seen visibility respect user settings.
- Private media is accessed through short-lived signed URLs.
- Define data export, deletion, retention, and moderation evidence policies before public launch.

14.4 Abuse prevention

- Rate-limit login, registration, username lookup, friend requests, message sends, comments, and uploads.
- Add spam heuristics and report workflows.
- Apply upload quotas and message throttles.
- Log security-relevant actions without logging passwords, tokens, or private message bodies unnecessarily.

14.5 Encryption scope

TLS protects data in transit. Database and object-storage encryption should be enabled at rest where supported. This is not end-to-end encryption. E2EE must be treated as a separate product and cryptographic architecture decision.

15. Non-functional Requirements and Reliability

Area	Requirement
Latency	Target near-real-time message delivery under normal network conditions; measure p50/p95/p99 in production.
Availability	Define an initial service objective before launch; use health checks and graceful degradation.
Durability	Persist messages before emitting message:new; use backups and tested restore procedures.
Consistency	Friend acceptance and friendship creation are transactional; message and outbox insert are transactional.
Scalability	Scale stateless API instances horizontally; use Redis adapter for Socket.IO when multiple instances are deployed.
Observability	Structured logs, metrics, traces where practical, error monitoring, and request IDs.
Accessibility	Keyboard navigation, focus visibility, semantic controls, and adequate contrast.
Compatibility	Support agreed modern desktop and mobile browsers; verify responsive layouts.

15.1 Database operations

- Use migrations; never edit production schema manually without a tracked migration.
- Run automated backups and periodically test restores.
- Monitor connection pool saturation, slow queries, locks, and index usage.
- Do not partition messages until measured data volume and query patterns justify it.

16. Testing and Quality Assurance

Layer	Suggested tools	Coverage
Unit	Jest	Services, policies, validators, domain rules.
API integration	Supertest + test database	Controllers, guards, database transactions.
Frontend	Vitest + Testing Library	Components, hooks, form behavior.
Browser E2E	Playwright	Registration, friendship, chat, moments, themes.
Load	k6	Message throughput, feed, concurrent sockets.
Security	Dependency scanning + manual tests	IDOR, auth, rate limits, upload validation.

16.1 Mandatory negative tests

- Non-friend attempts to create a direct conversation.
- Blocked user attempts to send a message or friend request.
- User guesses another conversation ID and requests its history.
- Non-friend requests a friends-only post or its media URL.
- Client tries to attach another user's media asset.
- Duplicate friend requests are submitted concurrently.
- Duplicate message retry is submitted concurrently.
- Expired/revoked refresh token is reused.
- Invalid MIME type or oversized media is uploaded.

16.2 Release gates

- All critical unit and integration tests pass.
- E2E flows pass on supported browsers.
- No unresolved critical/high security finding.
- Database migration and rollback/restore procedure are tested.
- Monitoring, backups, error reporting, and incident contacts are configured.

17. Development Roadmap and Milestones

Phase	Scope	Exit deliverable
0. Foundation	Monorepo, Next.js, NestJS, Docker, PostgreSQL, Prisma, Redis, CI	Web, API, and database run locally.
1. Identity	Registration, login, verification, sessions, profile, settings	Users can securely create and manage accounts.
2. Friendship	Exact username lookup, requests, acceptance, unfriend, block	Mutual friendship rules enforced.
3. Direct messaging	Conversation creation, Socket.IO, persistence, history, receipts, typing, presence	Reliable one-to-one chat.
4. Moments	Media pipeline, permanent posts, chronological friends feed, reactions, comments	Friends-only social sharing.
5. Expansion	Group chat, voice messages, advanced notification controls	Expanded communication experience.
6. Polish	Responsive UI, themes, accessibility, performance	Production-quality web client.
7. Release	Security, load tests, deployment, monitoring, backups	First production release.

17.1 First milestone checklist

1. Create Git repository and PNPM workspace.
2. Initialize Next.js web app and NestJS API.

3. Configure TypeScript, linting, formatting, and shared configuration.
4. Create Docker Compose for PostgreSQL and Redis.
5. Configure Prisma and baseline migration.
6. Add environment validation and secret handling.
7. Add /health and /ready endpoints.
8. Connect the web client to the API.
9. Add CI checks for lint, typecheck, and tests.
10. Document local setup in README.

18. Deployment and Operations

18.1 Environments

- Local: Docker Compose with PostgreSQL, Redis, and local S3-compatible storage such as MinIO.
- Staging: production-like deployment with test data and isolated secrets.
- Production: managed PostgreSQL, managed Redis, private object storage, TLS, backups, and monitoring.

18.2 Deployment components

Component	Deployment guidance
Next.js	Deploy as a web service or container; configure secure headers and caching.
NestJS API	Containerized stateless service behind a reverse proxy/load balancer.
Socket.IO	Same authenticated backend deployment initially; Redis adapter when horizontally scaled.
PostgreSQL	Managed service preferred; automated backups and point-in-time recovery if available.
Redis	Managed service with authentication and private network access.
Object storage	Private bucket, lifecycle rules, signed URLs.
Worker	Separate process/container for outbox, media processing, and push delivery.

18.3 Observability

- Structured JSON logs with requestId and userId where safe.
- Metrics for API latency, error rate, active sockets, message throughput, queue lag, database connections, and media failures.
- Error tracking for frontend and backend.
- Alerts for elevated error rate, queue backlog, database saturation, and failed backups.

19. Acceptance Criteria for v1

1. A user can register, verify, log in, log out, and manage a profile.
2. A user can find a known person by exact username and send a friend request.
3. A recipient can accept or reject a request; acceptance creates one mutual friendship.
4. Only accepted, non-blocked friends can create or send direct messages.
5. Messages persist and are recoverable after disconnect/reconnect.
6. Message delivery and read state are represented correctly.
7. A user can create permanent text/photo/short-video posts.
8. The feed contains only own posts and accepted friends' posts, newest first.
9. Non-friends cannot retrieve friends-only posts or media by guessing IDs.
10. Users can react to and comment on posts they are allowed to view.
11. Users can unfriend, block, and report.
12. Light and Dark themes work across supported screens.

13. The web application is responsive and accessible for core flows.
14. Production monitoring, backups, and restore procedures are operational.

20. Open Decisions Before Implementation

The following items are intentionally marked as decisions to confirm rather than silently assumed requirements.

Decision	Recommended starting point
Registration identifier	Email or phone; require at least one verified contact. Decide whether both are supported at launch.
Friend request message	Optional short introduction, rate-limited.
Video duration	Proposed 60 seconds maximum; confirm after storage/transcoding tests.
Media limits	Set image/video/voice size, resolution, codec, and quota limits after infrastructure selection.
Voice messages	Include after media pipeline is stable; otherwise Phase 2.
Group membership policy	Phase 2 decision: define inviter relationship, admin permissions, block/unfriend behavior, and history visibility.
Unfriend history	Recommended: retain existing history, disable new direct messages until re-friending.
Block history	Define whether existing conversation is hidden, read-only, or retained in an archive.
Message edit/delete	Define time window, tombstone behavior, and what recipients see.
Data retention	Define retention for deleted accounts, messages, reports, and media.
E2EE	Not in v1; evaluate separately before making any encryption promise.

Appendix A. Companion Files

- database/schema.sql — PostgreSQL baseline DDL with tables, constraints, and indexes.
- database/erd.mmd — Mermaid ER diagram source.
- api/openapi-outline.yaml — initial REST API contract outline.
- architecture/project-folder-structure.txt — monorepo directory layout.
- README.md — package overview and implementation cautions.

Appendix B. Glossary

Term	Meaning
Mutual friendship	A connection established only after a friend request is accepted.
Friends-only post	A permanent post accessible only to the author and currently accepted friends, subject to block rules.
Cursor pagination	Pagination based on a stable record cursor rather than page number.
Outbox pattern	Persisting an event in the same transaction as business data, then publishing asynchronously.
Presence	Transient online/last-seen state, usually held in Redis.
Signed URL	A time-limited URL granting access to a private object after authorization.
Modular Monolith	One deployable backend organized into clear domain modules.
E2EE	End-to-end encryption, where only communicating endpoints can decrypt message content.